

DÉON UI

A reusable React component library and interactive product demo for a modern perfume e-commerce experience.

Links

- **Live deployment:** [Showcase link](#)
- **GitHub repository:** [GitHub repository URL](#)

Project Overview

DÉON UI demonstrates Task 2: a functional, reusable component library built with React, TypeScript, and Tailwind CSS. The components are intentionally small and composable. They expose typed props, forward native HTML attributes where appropriate, and use consistent tokens for typography, color, spacing, focus states, and motion.

The demo page presents the components in a realistic perfume storefront context rather than as isolated examples. Visitors can browse products, open product details, add products to a cart, search the collection, open a cart drawer, adjust quantities, and submit a simulated checkout form.

Technology

- React 19
- TypeScript 6
- Vite
- Tailwind CSS 4
- Lucide React for interface icons
- Framer Motion for cart drawer transitions
- ESLint

Getting Started

```
npm install
npm run dev
```

The development server is usually available at <http://localhost:5173>.

Production checks

```
npm run build
npm run lint
npm run preview
```

Architecture

```
src/
├── components/
│   ├── Alert.tsx
│   ├── Badge.tsx
│   ├── Button.tsx
│   ├── CartDrawer.tsx
│   ├── CheckoutForm.tsx
│   ├── Input.tsx
│   ├── Loader.tsx
│   ├── Modal.tsx
│   ├── Navbar.tsx
│   ├── ProductCard.tsx
│   └── Search.tsx
├── data/
│   └── products.ts
├── pages/
│   └── ComponentDemo.tsx
├── App.tsx
├── index.css
└── main.tsx
```

- **components/** contains reusable UI primitives and composed commerce patterns.
- **data/** contains the typed product model and demo data.
- **pages/** contains the reference implementation that composes the library.
- **index.css** contains Tailwind theme tokens and global accessibility defaults.

Component Documentation

Button

Purpose: A typed action control for primary, secondary, outline, ghost, and destructive actions. It supports sizes, disabled state, native button attributes, and a built-in loading state.

Props:

- **variant?:** 'primary' | 'secondary' | 'outline' | 'ghost' | 'danger' (default: 'primary')
- **size?:** 'sm' | 'md' | 'lg' (default: 'md')
- **loading?:** boolean (default: false)
- **children:** ReactNode
- All native **ButtonHTMLAttributes<HTMLButtonElement>**, including **type**, **onClick**, **disabled**, and **aria-*** attributes.

Example:

```
<Button
  size="lg"
  variant="primary"
```

```
loading={isSubmitting}
onClick={handleSubmit}
>
  Continue to checkout
</Button>
```

Navbar

Purpose: Responsive site navigation with desktop links, a mobile menu, cart access, and a cart item count.

Props:

- `cartItemCount`: number
- `onOpenCart`: `() => void`

Example:

```
<Navbar
  cartItemCount={cartItems.length}
  onOpenCart={() => setCartOpen(true)}
/>
```

Badge

Purpose: Compact labels for product status and category metadata.

Props:

- `variant?: 'default' | 'outline' | 'dark'` (default: 'default')
- All native `HTMLAttributes<HTMLSpanElement>`, including `className`, `id`, and `aria-*` attributes.
- `children`: `ReactNode`

Example:

```
<Badge variant="dark">Bestseller</Badge>
```

ProductCard

Purpose: Displays a product image, brand, rating, description, price, status badge, and optional actions.

Props:

- `product`: `Product`
- `onAddToCart?: (product: Product) => void`
- `onViewDetails?: (product: Product) => void`

Example:

```
<ProductCard
  product={product}
  onAddToCart={addToCart}
  onViewDetails={setSelectedProduct}
/>
```

Input

Purpose: Labelled form input with required state, helper text, validation messaging, and optional leading content.

Props:

- `label?: string`
- `error?: string`
- `helperText?: string`
- `startAdornment?: ReactNode`
- All native `InputHTMLAttributes<HTMLInputElement>`, including `type`, `value`, `onChange`, `name`, `placeholder`, `required`, and `disabled`.

Example:

```
<Input
  label="Email address"
  type="email"
  placeholder="you@example.com"
  error={emailError}
  value={email}
  onChange={(event) => setEmail(event.target.value)}
  required
/>
```

Alert

Purpose: Communicates success, error, warning, and informational feedback. It can optionally render a dismiss action.

Props:

- `children: ReactNode`
- `variant?: 'success' | 'error' | 'warning' | 'info' (default: 'info')`
- `title?: string`
- `onClose?: () => void`

Example:

```
<Alert variant="success" title="Saved" onClose={dismissAlert}>
  Your fragrance was added to the collection.
</Alert>
```

Modal

Purpose: Accessible modal dialog with backdrop dismissal, Escape-key handling, body scroll locking, and a labelled title.

Props:

- `isOpen`: boolean
- `onClose`: () => void
- `title`: string
- `children`: ReactNode

Example:

```
<Modal
  isOpen={Boolean(selectedProduct)}
  onClose={() => setSelectedProduct(null)}
  title={selectedProduct?.name ?? 'Product details'}
>
  <ProductDetails product={selectedProduct} />
</Modal>
```

CartDrawer

Purpose: Animated shopping cart surface with empty state, quantity controls, item removal, subtotal calculation, and checkout loading state.

Props:

- `isOpen`: boolean
- `onClose`: () => void
- `items`: CartItemType[]
- `onUpdateQuantity`: (productId: number, newQuantity: number) => void
- `onCheckout`: () => void

`CartItemType` contains `product`: `Product` and `quantity`: `number`.

Example:

```
<CartDrawer
  isOpen={cartOpen}
  onClose={() => setCartOpen(false)}
  items={cartItems}
```

```
onUpdateQuantity={updateQuantity}  
onCheckout={completeCheckout}  
/>
```

CheckoutForm

Purpose: Reusable responsive shipping form with controlled submission loading state and native form validation.

Props: None currently. The form owns its simulated submission state internally.

Example:

```
<CheckoutForm />
```

Search

Purpose: Searchable product picker with debounced-feeling simulated network latency, cached queries, request cancellation, outside-click dismissal, loading state, and empty results state.

Props:

- `onSelectProduct: (product: Product) => void`

Example:

```
<Search onSelectProduct={({product}) => setSelectedProduct(product)} />
```

Loader

Purpose: Announces asynchronous work to assistive technology while displaying a size-based spinner.

Props:

- `size?: 'sm' | 'md' | 'lg' (default: 'md')`
- `label?: string (default: 'Loading')`

Example:

```
<Loader size="md" label="Loading recommendations" />
```

State Management Demonstration

The project uses local React state and callback props rather than adding a global state dependency for a small library demo.

- **ComponentDemo** owns selected product, cart count, email input, and toast visibility.
- **Navbar** owns mobile menu visibility and receives cart state through props.
- **Search** owns query, results, loading, dropdown visibility, cache, and request cancellation.
- **CartDrawer** owns checkout loading while the parent owns cart item data and quantity updates.
- **CheckoutForm** owns its submission state and prevents the browser's default submit behavior.
- **Modal** owns no business state; it receives open state and close behavior from its parent.

This separation keeps presentation components reusable while leaving business decisions with the consuming page.

Accessibility and Responsive Design

- Semantic buttons, links, forms, labels, and dialog roles are used throughout.
- Inputs connect labels, errors, and helper text with `htmlFor`, `aria-invalid`, and `aria-describedby`.
- Modal Escape handling and body scroll locking support keyboard and focused interaction.
- Loading states expose `aria-busy` or `role="status"` where appropriate.
- Tailwind responsive utilities support mobile, tablet, and desktop layouts.
- Focus-visible outlines and sufficient contrast are part of the shared component styling.
- Icon-only controls include accessible labels.

Task 2 Deliverables Checklist

- ☒ Functional React component library
- ☒ Reusable typed components with customization props
- ☒ Consistent naming and styling conventions
- ☒ Clean modular folder structure
- ☒ Basic state management demonstration
- ☒ Demo page showcasing the component system
- ☒ GitHub repository URL added above
- ☒ Live deployment URL added above

Reflection

Component Design Approach

My approach to building the DÉON UI library was deeply rooted in atomic design principles and modern headless UI concepts. Instead of hardcoding complex layouts, I abstracted the UI into foundational blocks (**Badge**, **Button**, **Input**). I utilized a strict, prop-driven architecture, explicitly defining **variants** and **sizes** using configuration dictionaries. This allows consuming developers to build interfaces rapidly (e.g., `<Button size="lg" variant="outline">`) while ensuring the luxury, monochromatic aesthetic remains perfectly consistent across the application.

Challenges Encountered

A significant challenge was balancing component flexibility with strict design constraints. For example, the **Input** component needed to handle standard text entry while gracefully accommodating **helperText** and **error** states without breaking vertical rhythm. Additionally, managing complex state in the **Search** component—specifically utilizing an **AbortController** to cancel stale requests and a **Map** ref to memoize results—required careful lifecycle management to prevent race conditions.

Ensuring Reusability

To guarantee reusability, I ensured all atomic components extended native React HTML interfaces (e.g., `interface InputProps extends React.InputHTMLAttributes<HTMLInputElement>`). This best practice ensures that standard HTML attributes (like `readOnly`, `disabled`, or `aria-labels`) can be passed natively without requiring custom prop mapping. Furthermore, using `React.forwardRef` on the `Input` ensures it can be seamlessly integrated with modern form libraries like React Hook Form.

Key Lessons Learned

This task reinforced the critical importance of Developer Experience (DX). A component library is only as effective as its API. By keeping props predictable and encapsulating complex internal logic (like Framer Motion animations in the `Modal`, or SVG spinners inside the `Button`'s loading state), the resulting library is highly maintainable, scalable, and ready for production environments.